

Error vs Exception



comparison between error and exception in java



In Java, **errors** and **exceptions** are two distinct types of issues that occur during program execution. Here's a comparison:

Aspect	Error	Exception
Definition	Errors are severe issues typically caused by the environment or system, such as lack of resources.	Exceptions are issues caused by the application code that can usually be caught and handled.
Hierarchy	Errors belong to the <code>Error</code> class and are part of the <code>java.lang</code> package.	Exceptions belong to the <code>Exception</code> class (which extends <code>Throwable</code>).
Recoverability	Errors are generally not recoverable. They indicate conditions that an application cannot handle (e.g., <code>OutOfMemoryError</code> , <code>StackOverflowError</code>).	Exceptions can often be recovered from if properly handled with a <code>try-catch</code> block.
Examples	Examples include <code>OutOfMemoryError</code> , <code>StackOverflowError</code> , <code>VirtualMachineError</code> .	Examples include <code>IOException</code> , <code>NullPointerException</code> , <code>SQLException</code> .
Handling	Errors are usually not handled by programs, as they indicate serious issues that may require application termination.	Exceptions can and should be handled using <code>try-catch</code> blocks or <code>throws</code> in method signatures.
Type	Errors are <code>unchecked</code> (not checked at compile time).	Exceptions can be checked (checked at compile time) or unchecked (<code>unchecked at compile time</code>).
Subcategories	Errors have no significant subcategories because they generally indicate critical failures.	Exceptions have two main subcategories: checked exceptions (e.g., <code>IOException</code>) and unchecked exceptions (e.g., <code>NullPointerException</code>).
Purpose	Represent system-level or resource-related problems beyond application control.	Represent application-level problems and logical errors that the developer can handle.

both under Throwable but Errors can't be handled

Key Points

1. **Error:** Indicates an issue that the Java runtime or system cannot reasonably handle.
2. **Exception:** Indicates a problem that the program may want to catch and handle gracefully to maintain normal flow.



are all exceptions and errors subclasses of throwable class as implied at <https://www.geeksforgeeks.org/exceptions-in-java/>



Yes, in Java, both **exceptions** and **errors** are subclasses of the `Throwable` class. This means that `Throwable` serves as the root of the exception-handling hierarchy.

Here is a brief explanation of how the class hierarchy is organized:

Hierarchy

1. `Throwable`

- The base class for all exceptions and errors in Java.
- It defines common behavior for both exceptions and errors, such as stack trace handling and detailed message storage.

java

```
public class Throwable {
    // Constructors, methods for stack trace, etc.
}
```

2. `Error` (subclass of `Throwable`)

- Represents serious issues that the application cannot handle (e.g., `OutOfMemoryError` or `StackOverflowError`).
- Example:

java

```
public class Error extends Throwable {
    // Specific error-related functionality
}
```

3. `Exception` (subclass of `Throwable`)

- Represents conditions that the application might want to catch and handle (e.g., `IOException`, `ArithmeticException`).
- Example:

java

```
public class Exception extends Throwable {
    // Specific exception-related functionality
}
```

4. Subcategories of Exceptions:

- **Checked Exceptions:** Exceptions that must be declared in the `throws` clause or handled explicitly using `try-catch` (e.g., `IOException`).
- **Unchecked Exceptions:** Subclasses of `RuntimeException`, which do not require explicit handling (e.g., `NullPointerException`, `ArrayIndexOutOfBoundsException`).

Simplified Hierarchical Diagram

```
plaintext

java.lang.Object
├── java.lang.Throwable
│   ├── java.lang.Error
│   └── java.lang.Exception
│       └── java.lang.RuntimeException
```

Key Point from GeeksforGeeks

The statement on GeeksforGeeks is correct: **all exceptions and errors are subclasses of `Throwable`**, enabling them to be used in exception-handling mechanisms, such as `try-catch` blocks or being propagated with the `throws` keyword.

However, the common practice is to handle exceptions only and let errors terminate the application, as errors are usually not recoverable.



explain "Errors belong to the Error class and are part of the java.lang package." in the first question according to <https://www.geeksforgeeks.org/exceptions-in-java/>



The statement "Errors belong to the Error class and are part of the java.lang package" means that **Errors** in Java are a specific type of issue that is represented by the `Error` class, which is a part of the `java.lang` package in the standard Java library.

Here's a detailed explanation in the context of Java's exception hierarchy and the explanation on GeeksforGeeks:

1. The `Error` Class

- The `Error` class is a direct subclass of `Throwable`.
- It represents serious problems that a reasonable application cannot handle and typically indicates issues that are related to the environment in which the application is running (such as the JVM itself).

```
java

public class Error extends Throwable {
    // Error-specific properties and methods
}
```

2. Location in `java.lang` Package

- The `java.lang` package is a core package in Java that provides fundamental classes such as `Object`, `String`, `Thread`, `Throwable`, and others.

- Since `Error` is part of `java.lang`, it is automatically available in every Java program without needing explicit imports.

3. Examples of Subclasses of `Error`

The `Error` class has several commonly encountered subclasses, all of which are part of the `java.lang` package:

- `OutOfMemoryError`: Occurs when the Java Virtual Machine (JVM) cannot allocate enough memory.
- `StackOverflowError`: Occurs when the stack space is exhausted, often due to deep or infinite recursion.
- `VirtualMachineError`: Indicates critical errors in the JVM itself (e.g., internal errors or inability to execute further instructions).

```
java

try {
    // Code that causes a stack overflow error
    recursiveCall();
} catch (StackOverflowError e) {
    System.out.println("StackOverflowError caught!");
}
```

4. Relationship with `Throwable`

- Both `Error` and `Exception` classes derive from `Throwable`.
- This allows both types of issues to be thrown using the `throw` statement and managed with `try-catch` blocks, though it is not recommended to catch `Error` instances because they typically indicate unrecoverable conditions.

```
plaintext

java.lang.Object
├─ java.lang.Throwable
│   └─ java.lang.Error
│       ├── java.lang.OutOfMemoryError
│       └─ java.lang.StackOverflowError
```

Key Takeaway:

As correctly noted on GeeksforGeeks, `Error` and its subclasses are part of the `java.lang` package, emphasizing their foundational role in Java's error-handling mechanism. However, they represent issues that are usually beyond the scope of application handling and should generally be left for the JVM or system to deal with.